The numpy package

Ezequiel López-Rubio

Department of Computer Languages and Computer Science

University of Málaga, Spain

Contents

- Introduction
- Fundamentals
- Array access
- Operations
- Conclusion

1. INTRODUCTION

1. Introduction

Overview

- The numpy package is the base for scientific computation in Python
- The main feature of numpy is the ndarray class
  - It represents n-dimensional arrays of a single type
  - Most operations are optimized for performance by vectorization and pre-compilation
  - A wide range of mathematical operations are provided for them

1. Introduction

Main differences

- The main differences between ndarrays and standard Python sequences are:
  - ndarrays have a fixed size at creation
  - All elements in a ndarray must be of the same data type, except for ndarrays of generic objects
  - Many built-in numpy operations accept Python sequences as inputs but they are converted to ndarrays prior to processing

2. FUNDAMENTALS

2. Fundamentals

Importing and base data types

- The usual way to import the package is:

import numpy as np

- Some base data types that a ndarray can hold:
  - numpy.bool_ (Boolean values stored as bytes, True or False)
  - numpy.int16, numpy.int32, numpy.int64 (signed integers)
  - numpy.uint16, numpy.uint32, numpy.uint64 (unsigned integers)
  - numpy.float16, numpy.float32, numpy.float64 (floating point numbers)

2. Fundamentals

The ndarray class

- The ndarray class is also known by the alias array
- The most important attributes of an ndarray object are:
  - The number of axes (dimensions): ndarray.ndim
  - The shape, i.e., a tuple of integers indicating the size in each dimension: ndarray.shape
  - The total number of elements: ndarray.size
  - The type of the elements in the array: ndarray.dtype

2. Fundamentals

Array creation

- From a regular Python list or tuple: numpy.array

```
>>> a = np.array([5, 8, -29])
```

- Sequences of sequences are converted to two-dimensional ndarrays

```
>>> b = np.array([(1.5, 2, 3), (4, 5, 6)])
```

- The array type can be specified at creation time:

```
>>> c = np.array([[1, 2], [3, 4]], dtype=np.float64)
```

2. Fundamentals

Array creation

- If the size of the array is known at creation time but its contents are not:

```
>>> d = np.zeros((3, 4))
>>> e = np.ones((5, 6, 2), dtype=np.int32)
>>> f = np.empty((6, ))
```

- It is possibly to create a new array with the same shape and data type as a given array:

```
>>> g = np.zeros_like(e)
```

- Array copy:

```
>>> h = np.copy(g)
```

2. Fundamentals

Array creation

- To create sequences of integers, the numpy.arange function is analogous to the Python built-in range but returns an array:

```
>>> np.arange(20, 50, 3)
array([20, 23, 26, 29, 32, 35, 38, 41, 44, 47])
```

- To create sequences of floating-point numbers, the numpy.linspace function is recommended:

```
>>> np.linspace(0, 2, 9)
array([0.  , 0.25, 0.5 , 0.75, 1.  , 1.25, 1.5 , 1.75, 2.  ])
```

2. Fundamentals

Printing arrays

- One dimensional arrays are printed as rows, bidimensional arrays as matrices, and tridimensional arrays as lists of matrices

```
>>> a = np.arange(6)
>>> print(a)
[0 1 2 3 4 5]
>>> b = np.arange(12).reshape(4, 3)
>>> print(b)
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

2. Fundamentals

Basic operations

- Arithmetic operations are applied elementwise: +, -, *, /, //, **
- The mathematical matrix product operator is @
- The matrix transpose is np.ndarray.T
- Some operations like += and *= modify the existing array rather than creating a new one
- If arrays of different types are operated, the resulting array has the most general/precise type (upcasting)
- Most unary operations are methods of the ndarray class: sum, min, max, argmin, argmax
  - They apply to the flattened version of the array unless the axis parameter is specified

2. Fundamentals

Broadcasting

- When arrays with different shapes are encountered during arithmetic operations, the smaller array is broadcast across the larger array so that they have compatible shapes
  - Missing dimensions are assumed to have size one
- This usually results in faster vectorized operations

```
>>> a = np.array([1.0, 2.0, 3.0])
>>> b = 2.0
>>> a * b
array([2.,  4.,  6.])
```

3. ARRAY ACCESS

3. Array access
Indexing

- Arrays are indexed with the x[obj] syntax, where x is the array and obj the selection. There are two kinds of indexing depending on obj:
  - Basic indexing, which returns a view. It works like standard Python indexing. It is activated when obj is an integer, a slice constructed by start:stop:step , or a tuple of slices and integers
  - Advanced indexing, which returns a copy. It is triggered when obj is a non-tuple sequence, an ndarray of type integer or bool, or a tuple with at least one sequence or ndarray of type integer or bool

3. Array access

Basic indexing

- Single element indexing works the same that standard Python sequences:
  - Zero based indices
  - Negative indices are accepted (from the end of the array)
- The indices of the dimensions form a tuple:

a[4, 5, -1] == a[(4, 5, -1)]

- If there are fewer indices than dimensions, a subdimensional array view (not a copy) is obtained
- It is possible but inefficient to index the subdimensional array:
- b[4, 5] == b[4][5]

3. Array access
Basic indexing

- Unlike Python lists, you can never grow an array
- There are two valid possibilities to perform an assignment x[obj] = value:
  - The value is the same shape as x[obj]
  - The value can be replicated (broadcast) to be the same shape as x[obj]
  - All other cases fail with an execution error
- The ellipsis (…) expands to the number of : objects needed for the selection tuple to index all dimensions

x.shape == (2,3,4) => x[…, 1] == x[:, :, 1]

- Each np.newaxis object in the selection tuple serves to insert a unit-length dimension. It is an alias of None:

x[:, np.newaxis, :, :].shape == x[:, None, :, :].shape == (2, 1, 3, 4)

3. Array access

Advanced indexing

- It always returns a copy of the data as opposed to basic indexing that returns a view
- Integer arrays can be employed to select arbitrary items
  - Each integer array represents some indices into an array dimension
  - Negative indices are allowed
  - All integer arrays must have the same shape, or be broadcast to the same shape
  - Broadcasting is used when integer arrays are combined with scalars for other dimensions
- Boolean arrays can also be employed to select items
  - The Boolean array should be the same shape as the indexed array

4. OPERATIONS

4. Operations

Array reshape and creation

- Arrays can be changed to the desired shape with numpy.reshape:

```
>>> a = np.arange(6).reshape((3, 2))
>>> a
array([[0, 1],
       [2, 3],
       [4, 5]])
```

- Some utilities are provided to create specific arrays:
  - An array full of a given value with numpy.full and numpy.full_like
  - A bidimensional array that represents a unit matrix with numpy.eye
  - A bidimensional array with all zeros except the main diagonal with numpy.diag

4. Operations

Array modification

- Some utilities are provided to modify an array
- A new array with the requested changes is created, i.e., the original array is not modified
- Subarrays along an axis can be deleted with numpy.delete
- Values can be inserted along an axis with numpy.insert
- Values can be appended to the end of an array with numpy.append

4. Operations
Array modification

```
>>> a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
>>> np.delete(a, 1, 0)
array([[1, 2, 3],
       [7, 8, 9]])
>>> np.insert(a, 1, [20, 21, 22], 0)
array([[ 1,  2,  3],
       [20, 21, 22],
       [ 4,  5,  6],
       [ 7,  8,  9]])
>>> np.append(a, [[20, 21, 22]], 0)
array([[ 1,  2,  3],
       [ 4,  5,  6],
       [ 7,  8,  9],
       [20, 21, 22]])
```

4. Operations

Array combination and repetition

- A sequence of arrays can be joined along an axis with numpy.concatenate
- Arrays can be stacked horizontally (column wise) with numpy.hstack
- Arrays can be stacked vertically (row wise) with numpy.vstack
- An array can be tiled several times with numpy.tile
- Each element of an array can be repeated with numpy.repeat

4. Operations

Array combination and repetition

```
>>> a = np.array([[1, 2], [3, 4]])
>>> b = np.array([[5, 6], [7, 8]])
>>> np.concatenate((a, b), axis=0)
array([[1, 2],
       [3, 4],
       [5, 6],
       [7, 8]])
>>> np.hstack((a, b))
array([[1, 2, 5, 6],
       [3, 4, 7, 8]])
```

4. Operations
Array combination and repetition

```
>>> np.vstack((a, b))
array([[1, 2],
       [3, 4],
       [5, 6],
       [7, 8]])
>>> np.tile(a, 2)
array([[1, 2, 1, 2],
       [3, 4, 3, 4]])
>>> np.repeat(a, 2, axis=1)
array([[1, 1, 2, 2],
       [3, 3, 4, 4]])
```

4. Operations

Comparisons and logical operations

- Comparisons and logical operations are carried out elementwise unless otherwise specified: <, >, <=, >=, ==, !=, &, |, np.logical_not
- Arraywise comparisons: np.array_equal (same shape and elements), np.array_equiv (can be broadcast to the same shape and elements)
- Truth value testing: np.all (all array elements along an axis evaluate to True), np.any (any array element along an axis evaluates to True)

4. Operations

Comparisons and logical operations

```
>>> x = np.array([4, 0, 6, 9, 7, -5, 2])
>>> (x < 5) | (x >= 9)
array([ True,  True, False,  True, False,  True,  True])
>>> np.array_equiv(np.array([[3, 4], [3, 4]]), np.array([[3, 4]]))
True
>>> np.all( x < 20 )
True
>>> np.any( x < -30 )
False
```

4. Operations
Finding nonzeros, unique elements and sorting

- The indices in each dimension of nonzero elements can be retrieved with np.nonzeros
  - The count of nonzero elements is given by np.count_nonzero
- The unique elements of an array are found by np.unique
- A sorted copy of an array along an axis can be obtained with np.sort
  - If the original array must be modified, then np.ndarray.sort must be employed

4. Operations

Finding nonzeros, unique elements and sorting

```
>>> x = np.array([[3, 0, 0], [0, 4, 0], [5, 6, 0]])
>>> np.nonzero(x)
(array([0, 1, 2, 2]), array([0, 1, 0, 1]))
>>> np.count_nonzero(x)
4
>>> np.unique(x)
array([0, 3, 4, 5, 6])
>>> np.sort(x, axis=0)
array([[0, 0, 0],
       [3, 4, 0],
       [5, 6, 0]])
```

4. Operations

Random array generation

- Array of random integers: np.random.randint
- Array of random floats in [0.0, 1.0): np.random.random
- Array of random floats in a given interval: np.random.uniform
- Array of samples from a given unidimensional array: np.random.choice
- Array of random floats from the Gaussian (normal) distribution: np.random.normal

4. Operations
Random array generation

```
>>> np.random.randint(5, 9, size=(4,))
array([6, 6, 8, 7])
>>> np.random.random(size=(1, 5))
array([[0.76194304, 0.40986079, 0.12832288, 0.8587626 , 0.27057258]])
>>> np.random.uniform(2.3, 8.5, size=(1, 4))
array([[7.47790829, 6.52973762, 3.65584661, 3.94379477]])
>>> np.random.choice(["BCN", "MAD", “AGP", "ZAZ"], size=(3,))
array([‘BCN', ‘AGP', ‘BCN'], dtype='<U9’)
>>> np.random.normal(1.5, 0.02, size=(4,))
array([1.48366031, 1.49799955, 1.49129494, 1.50944283])
```

4. Operations

Universal functions

- They operate on arrays elementwise
- They support broadcasting
- They are vectorized for speed
- Floating point functions: np.round (round to the nearest integer), np.floor (round to np.Inf), np.ceil (round to -np.Inf), np.trunc (truncation)
- Arithmetic functions: np.add, np.subtract, np.multiply, np.divide
- Transcendental functions: numpy.sqrt, numpy.log, numpy.exp
- Trigonometric functions: np.sqrt, np.sin, np.cos, np.tan, np.arctan, np.arctan2

4. Operations

Linear algebra

- Determinant of a matrix: np.linalg.det
- Inverse of a matrix: np.linalg.inv,
- Matrix power: np.linalg.matrix_power
- Solve a linear system of equations: np.linalg.solve

```
>>> a = np.array([[1, 2], [3, 5]])
>>> b = np.array([1, 2])
>>> x = np.linalg.solve(a, b)
>>> x
array([-1.,  1.])
>>> x2 = np.linalg.inv(a) @ b
>>> x2
array([-1.,  1.])
```

4. Operations

Statistics

- Sum/product along an axis: np.sum / np.prod
- Mean/median/standard deviation along an axis: np.mean, np.median, np.std
- Quantile along an axis: np.quantile

```
>>> np.mean(np.random.random(size=(100,)))
0.4772436945010195 6
>>> np.quantile(np.random.random(size=(100,)), 0.6)
0.6184713332879888
```

5. CONCLUSION

5. Conclusion

- The numpy package forms the foundation for most scientific and engineering software written in Python
- Indexing and broadcasting must be employed with care so as to obtain the intended results
- Loops should be avoided at all costs in order to boost the performance of the code by exploiting the vectorized implementations provided by numpy
- Unnecessary array copies should also be avoided
- Overflow, underflow and roundoff errors must be appropriately managed